

الخطة التفصيلية لمشروع المنصة التعليمية الذكية (RAG & OCR)

بناءً على ملف توثيق المشروع (Rag Ocr Project Documentation Ar.pdf) ومتطلباتك الإضافية، تم إعداد هذه الوثيقة الشاملة لتوضيح كيفية تنفيذ المرحلة الأولى، التقنيات المستخدمة، وكيفية تلبية متطلبات الأمان، السرعة، والضغط العالي.

1. تفاصيل المرحلة الأولى (ال RAG) وكيفية التنفيذ

المرحلة الأولى ستركز حصرياً على الـ **RAG** للتعامل مع ملفات PDF و Word.

الخطوات التي سيقوم بها النظام (Backend Pipeline):

1. **استقبال الملف**: يقوم المستخدم برفع الملف عبر الواجهة.
2. **فحص الملف أمنياً**: للتأكد من خلوه من الفيروسات (سيتم تفصيل ذلك لاحقاً).

3. **استخراج النص (Parsing)**: باستخدام `PyMuPDF` للـ PDF و `python-docx` للـ Word.

4. **تنظيف وتقسيم النص (Chunking)**: تنظيف النص العربي والتقسيم الذكي بناءً على العناوين والفقرات لضمان عدم ضياع السياق.
5. **التحويل إلى متجهات (Embeddings)**:

- **الخيار المجاني/المفتوح**: استخدام `BGE-M3` (ممتاز جداً

ومتعدد اللغات).

- **الخيار المدفوع**: استخدام `OpenAI text-embedding-3-small` (سريع جداً، دقيق، وتكلفته شبه معدومة).

- **التوصية**: يُفضل استخدام المدفوع (`OpenAI`) لأنه لا يتطلب

سيرفرات قوية (GPU) لتشغيله، مما يوفر تكلفة الاستضافة.

6. **التخزين والاسترجاع (Vector DB & Hybrid Search)**: تخزين النصوص والبحث عنها وقت سؤال المستخدم.

متى تستخدم API مدفوع ومتى تستخدم مجاني؟

- النماذج اللغوية (LLMs - للإجابة على الأسئلة):

- **المدفوع**: مثل `GPT-4o-mini` أو `Claude 3.5 Haiku`. يُنصح

به بشدة في هذا المشروع للحصول على **سرعة فائقة** ودقة عالية في فهم اللغتين العربية والإنجليزية.

- **المجاني (المحلي)**: مثل `Llama 3`. يمكن استخدامه، ولكنه

يحتاج إلى سيرفرات ذات مواصفات عالية (GPU) والتي ستكون **مكلفة جداً** شهرياً أكثر من الـ APIs المدفوعة.

2. نظام تسجيل الدخول وصلاحيات الإدارة (SSO & Admin)

تسجيل دخول المستخدمين (بدون كلمات مرور)

لتوفير تجربة مستخدم سلسة وأمنة، سيتم الاعتماد على تسجيل الدخول

الموحد (SSO) باستخدام **Google** و **Microsoft**.

- **التقنية المقترحة**: استخدام مكتبة `NextAuth.js` (أو `Clerk`).

- **المميزات

- مجانية تماماً للمشاريع الصغيرة والمتوسطة.

- لا توجد كلمات مرور يمكن اختراقها أو نسيانها.

- أمان عالي وحماية من الحسابات الوهمية.

لوحة تحكم الإدارة (Admin Dashboard)

سيتم إضافة حقل في قاعدة البيانات `role: admin / user`.

- عندما يقوم الأدمن بتسجيل الدخول، ستظهر له لوحة تحكم خاصة تمكنه من:

1. مراقبة المحتوى المرفوع من قبل المستخدمين.

2. متابعة معدل الاستخدام (Usage Analytics).

3. حظر أو تقييد المستخدمين المخالفين.

3. الحماية وفحص الملفات المرفوعة (Antivirus)

لضمان عدم قيام المستخدمين برفع ملفات ملغمة، فيروسات، أو سكربتات خبيثة:

- **التقنية المقترحة (مجانية)**: `ClamAV`. هو محرك مكافحة فيروسات مفتوح المصدر ومجاني.

- **كيفية العمل**: عند رفع الملف، وقبل أن يتم حفظه أو معالجته في النظام، يتم تمريره إلى خدمة `ClamAV` في الخلفية. إذا تم اكتشاف أي تهديد، يتم رفض الملف فوراً وإشعار المستخدم (والأدمن).

- **حماية إضافية**: التحقق من نوع الملف الحقيقي (MIME Type Verification) لضمان أن الملف الذي امتداده `pdf` هو فعلاً ملف PDF وليس ملف تنفيذي `exe`. تم تغيير اسمه.

4. ضمان سرعة الرد الفائقة (أقل من ثوانٍ معدودة)

بما أن النظام سيستخدم في اختبارات وقتها ضيق (20 سؤال في 20 دقيقة)، السرعة تعتبر عاملاً حرجاً:

1. **استخدام نماذج سريعة (Fast LLMs)**: استخدام `GPT-4o-mini` من OpenAI أو استخدام نماذج مفتوحة عبر منصة `Groq` (التي تستخدم معالجات LPU توفر سرعة خيالية تصل لـ 800 كلمة في الثانية).

2. **البث المباشر للإجابة (Streaming)**: بدلاً من انتظار النموذج حتى يكتب الإجابة كاملة (مما قد يأخذ 5-10 ثوانٍ)، سيتم بث الإجابة كلمة بكلمة (مثل ChatGPT). المستخدم سيرى الرد يبدأ فوراً في أقل من ثانية.

3. **الاسترجاع السريع (Fast Vector DB)**: استخدام قاعدة بيانات متجهات سريعة جداً مثل `Pinecone` (مدفوعة ورخيصة) أو `Qdrant`.

5. التعامل مع الضغط العالي (+20 مستخدم في نفس الوقت)

لضمان عدم انهيار السيرفر عندما يدخل عدد كبير من المستخدمين في نفس الوقت ورفعوا ملفاتهم:

1. **معالجة الملفات في الخلفية (Background Processing)**:

- **المشكلة**: استخراج النص من PDF وتحويله لمتجهات عملية ثقيلة قد توقف السيرفر (Blocking).

- **الحل**: استخدام `Celery` مع `Redis` (تقنية مجانية). عندما يرفع المستخدم ملف، يتم وضع الطلب في "طابور" ويتم إخبار المستخدم

"جاري المعالجة..." يقوم السيرفر بمعالجة الملفات براحته دون أن ينهار، ثم يرسل إشعاراً عند الانتهاء.

2. **التوسع التلقائي (Auto-scaling)**:

- بدلاً من استئجار سيرفر واحد ثابت، سيتم استضافة ال Backend على منصات سحابية مثل `Google Cloud Run` أو `AWS App Runner` أو `Render`.

- **الميزة**: إذا كان هناك مستخدم واحد، يعمل سيرفر واحد. إذا دخل 20 أو 100 مستخدم فجأة، تقوم المنصة بنسخ السيرفر تلقائياً لتوزيع الحمل، وتعود لطبيعتها عند انتهاء الضغط (الدفع مقابل الاستخدام الحقيقي فقط).

6. قواعد البيانات المستخدمة

سيحتاج النظام إلى 3 أنواع من قواعد البيانات (جميعها يمكن البدء بنسخ مجانية منها):

1. **قاعدة بيانات علائقية (Relational DB)**:

- النوع: `PostgreSQL` (الأفضل والأكثر استقراراً).

- الاستخدام: تخزين بيانات المستخدمين، صلاحيات الأدمن، سجل الأسئلة والإجابات، والملفات المرفوعة.

2. **قاعدة بيانات متجهات (Vector DB)**:

- النوع: `Pinecone` (Cloud) أو `Qdrant` أو `Weaviate`.

- الاستخدام: تخزين ال Embeddings للبحث السريع داخل

الملفات.

3. **قاعدة بيانات مؤقتة (In-Memory DB)**:

- النوع: `Redis`.

- الاستخدام: إدارة طابور معالجة الملفات (Celery)، والتخزين

المؤقت (Caching) لتقليل الضغط على قاعدة البيانات الرئيسية وتخزين جلسات المستخدمين المفتوحة.

الخلاصة والتوصية الذهبية

لتحقيق متطلباتك (سرعة عالية، أمان، تحمل ضغط، ودقة):

- الواجهة: `Next.js` مع `NextAuth` لدخول جوجل/مايكروسوفت.

- الخلفية: `FastAPI` (بايثون) + `Celery` لمعالجة الملفات

وتجنب سقوط السيرفر.

- قواعد البيانات: `Redis` + `Pinecone` + `PostgreSQL`.

- النماذج الذكية (AI): استخدام APIs مدفوعة لكن رخيصة جداً

وسريعة (`GPT-4o-mini` للردود و `text-embedding-3-small`

للمتجهات). هذا سيجنبك دفع آلاف الدولارات شهرياً على استئجار

سيرفرات ببطاقات رسومية (GPUs) قوية.

- الأمان: فحص الملفات المرفوعة عبر `ClamAV`.

- البدء: نفذ خطة ال RAG أولاً بالكامل، واختبرها مع 20 مستخدم

في نفس الوقت. بعد التأكد من الاستقرار، انتقل لمرحلة ال OCR

لاحقاً كما تم التوضيح في ملف المشروع الأصلي.